A Bayesian uncertainty for `HEFTNet`

The mixture of linearised Laplace approximations, explained

With the code in `dihiggs/laplace.py`, `dihiggs/counts.py` and `code/heft_laplace.ipynb`

October 31, 2026

Abstract

This note explains, step by step and for readers who have not met the method before, how we approximate the posterior density of the parameters of `HEFTNet` by a mixture of Gaussians, one about each network of the fine-tuned ensemble, and how that gives a Bayesian uncertainty for the predicted cross section. It covers the noise model (the POWHEG spectra are scaled Poisson counts), the linearisation of a network and the Gaussian posterior it implies, MacKay's evidence approximation for the prior, a property of `HEFTNet` that makes the calculation exact and fast, the mixture, the model error, and the checks. The last section gives the first results. The plan that this implements is `docs/bnn/bnn_plan2.pdf`.

Contents

1 Why we are doing this **2**

2 The data are scaled event counts **2**

3 The Laplace approximation **2**
3.1 The idea . . . 2
3.2 Linearising the network . . . 3
3.3 The posterior and the prediction . . . 3

4 The prior: MacKay's evidence approximation **4**

5 The structure of HEFTNet, and why it makes everything exact and fast **4**

6 The mixture **5**

7 The model error **5**

8 Checks **6**

9 First results **6**

10 The code **7**

1 Why we are doing this

Bayes' theorem gives the density of the network parameters $\boldsymbol{\omega}$ given the training data $\mathcal{D}$,

$$p(\boldsymbol{\omega} \mid \mathcal{D}) \propto p(\mathcal{D} \mid \boldsymbol{\omega})\, \pi(\boldsymbol{\omega}), \tag{1}$$

and the uncertainty of a prediction follows by propagating that density through the network. Our Markov chains (Hamiltonian Monte Carlo, following Neal) gave widths of only 0.5% of the POWHEG noise. `bnn_plan2.pdf` shows why: a single chain of 10^3 iterations could not explore a 3,944-dimensional density whose curvatures differ by a factor 10^6 or more, and it stayed near one of the many equally good solutions. The approach here avoids sampling altogether. It replaces the exact posterior by an approximation that can be computed with linear algebra, and it builds in the several solutions by using the 20 networks of the ensemble.

2 The data are scaled event counts

Each POWHEG spectrum is a histogram of 40,000 generated events (about 16,000 for two short runs), all with the same weight q_s (in pb) for spectrum s. The cross section in bin k is

$$t_k = q_s\, n_k, \qquad n_k \in \{0, 1, 2, \ldots\}. \tag{2}$$

`dihiggs/counts.py` recovers q_s and the integers n_k of every spectrum exactly (`docs/counts/counts_note.pdf`). The count n_k is Poisson distributed, so

$$\mathrm{Var}(t_k) = q_s^2\, \mathrm{Var}(n_k) = q_s^2\, \langle n_k \rangle \approx q_s\, t_k. \tag{3}$$

We use $\sigma_k^2 = q_s(t_k + q_s)$: the added q_s^2 is the variance of one event, and it keeps the variance of empty or nearly empty bins from vanishing. The bins of different spectra, and (up to a negligible multinomial correlation) the bins of one spectrum, are independent. The 1,814 empty bins, which had been dropped from the training file, are restored: an empty bin tells us the cross section there is small.

Why not $q_s f_k$, the Poisson variance at the prediction?

For a Poisson likelihood the textbook weight of a bin is $1/(q_s f_k)$, with f_k the prediction. We tried it. It gives a huge weight to the few bins where a network predicts a negative or tiny value against a large count: in the first network, two of 274,152 rows contributed a quarter of the χ^2. Those bins would dominate the curvature in their directions. The networks were fine-tuned with $q_s(t_k + q_s)$, so we use it here too (switch `VARIANCE_FROM`).

3 The Laplace approximation

3.1 The idea

Near its maximum, almost any smooth, single-peaked density looks like a Gaussian: expand $\ln p$ to second order about the peak and exponentiate. This is the Laplace approximation. For a network, the expansion is done most usefully not in $\ln p$ directly but in the network itself.

3.2 Linearising the network

Let $\hat{\omega}$ be a trained network and $j(x) = \partial f(x;\omega)/\partial\omega$ at $\hat{\omega}$, a vector with one entry per parameter (3,944 for `HEFTNet`). For ω near $\hat{\omega}$,

$$f(x;\omega) \approx f(x;\hat{\omega}) + j(x)^{\mathsf{T}}(\omega - \hat{\omega}). \tag{4}$$

The right-hand side is *linear in the parameters*: it is a linear regression whose "basis functions" are the components of $j(x)$. Everything that follows is the textbook solution of a linear regression with Gaussian noise and a Gaussian prior.

A linear regression refresher

Fit $y_k = \phi_k^{\mathsf{T}}\beta + \text{noise}$, noise variance σ_k^2, with a Gaussian prior of precision matrix T on β. The log posterior is $-\frac{1}{2}\sum_k (y_k - \phi_k^{\mathsf{T}}\beta)^2/\sigma_k^2 - \frac{1}{2}\beta^{\mathsf{T}}T\beta$, a quadratic in β. The posterior is therefore exactly Gaussian, with precision (inverse covariance)

$$H = \sum_k \frac{\phi_k \phi_k^{\mathsf{T}}}{\sigma_k^2} + T.$$

The prediction $\phi^{\mathsf{T}}\beta$ at a new point is Gaussian with variance $\phi^{\mathsf{T}}H^{-1}\phi$. Each data point adds a "rank-one" piece $\phi_k\phi_k^{\mathsf{T}}/\sigma_k^2$: more data, or less noisy data, make H larger and the posterior narrower.

3.3 The posterior and the prediction

With $\phi_k = j_k \equiv j(x_k)$, a likelihood $\prod_k \mathcal{N}(t_k; f_k, \sigma_k^2)$ and Neal's prior, which gives each parameter i in group $g(i)$ (one group per weight matrix or bias vector, 16 for `HEFTNet`) a Gaussian of precision $\tau_{g(i)}$, the posterior of the linearised network is Gaussian about $\hat{\omega}$ with precision

$$H = G + T, \qquad G = \sum_{k=1}^{N} \frac{j_k j_k^{\mathsf{T}}}{\sigma_k^2}, \qquad T = \operatorname{diag}(\tau_{g(i)}). \tag{5}$$

G is called the (generalised) Gauss–Newton matrix. It is the curvature of $\chi^2/2$, except that a term proportional to the residuals $f_k - t_k$ is left out; that term is small near a good fit, and leaving it out guarantees that G has no negative eigenvalues. The prediction at x is Gaussian with

$$\text{mean } \mu(x) = f(x;\hat{\omega}), \qquad \text{variance } v(x) = j(x)^{\mathsf{T}}H^{-1}j(x). \tag{6}$$

This is MacKay's original treatment of Bayesian networks (1992), now called the "linearised Laplace approximation" or the "GLM predictive". Note that the network is linearised in the *prediction* as well as in the posterior; that consistency is what makes the method work well.

How the code computes v without inverting H

H is symmetric and positive definite, so it has a Cholesky factorisation $H = LL^{\mathsf{T}}$ with L lower triangular. Then $v = j^{\mathsf{T}}(LL^{\mathsf{T}})^{-1}j = |L^{-1}j|^2$, and $L^{-1}j$ is one triangular solve. For 3,944 parameters the factorisation takes about a second. Everything is done in double precision, because the eigenvalues of H span about 13 orders of magnitude.

4 The prior: MacKay's evidence approximation

The precisions τ_g control how far the parameters of group g may stray from zero. In the Markov chains they were sampled. Here we choose them by maximising the *evidence*, the probability of the data given τ with the parameters integrated out, which the Laplace approximation makes available in closed form. Setting its derivative with respect to τ_g to zero gives MacKay's fixed-point equations:

$$\tau_g = \frac{\gamma_g}{\sum_{i\in g} \hat{\omega}_i^2}, \qquad \gamma_g = n_g - \tau_g \sum_{i\in g} \left(H^{-1}\right)_{ii}, \tag{7}$$

with n_g the number of parameters in group g.

What γ means

Along an eigenvector of G with eigenvalue λ, the data contribute curvature λ and the prior τ. If $\lambda \gg \tau$ the data determine that combination of parameters; if $\lambda \ll \tau$ the prior does. $\gamma = \sum \lambda/(\lambda+\tau) = \mathrm{tr}(GH^{-1})$ counts the combinations the data determine: MacKay's "number of well-determined parameters". γ_g is its share in group g. The first equation of eq. (7) then says: the prior variance $1/\tau_g$ should equal the mean square of the *well-determined* parameters of the group.

The equations are solved by iteration. Only the diagonal of H changes with τ, so each iteration is one Cholesky factorisation. The plain iteration overshoots and oscillates, so each step moves half-way, in $\ln\tau$ (`damping = 0.5`). The expansion point $\hat{\omega}$ is held fixed; strictly it would move a little as τ changes, but the prior term is tiny beside the χ^2.

5 The structure of HEFTNet, and why it makes everything exact and fast

`HEFTNet` is not a generic network. Its output is

$$f(\boldsymbol{x};\boldsymbol{\omega}) = \sum_{i=1}^{23} C_i(\theta)\, a_i(m_{hh};\boldsymbol{\omega}), \tag{8}$$

where the C_i are the known polynomials in the five HEFT parameters θ and the 23 functions a_i are the outputs of the network, *which sees only* m_{hh}. Hence

$$\boldsymbol{j}(\boldsymbol{x}) = \sum_i C_i(\theta)\, \frac{\partial a_i(m_{hh})}{\partial \boldsymbol{\omega}} = A(m_{hh})^{\mathsf{T}}\, \boldsymbol{C}(\theta), \tag{9}$$

with $A(m_{hh})$ the $23 \times 3{,}944$ Jacobian of the 23 functions. There are only 80 distinct values of m_{hh} (the bins). Writing A_b for bin b,

$$G = \sum_b A_b^{\mathsf{T}} S_b A_b, \qquad S_b = \sum_{k\in \mathrm{bin}\ b} \frac{\boldsymbol{C}_k \boldsymbol{C}_k^{\mathsf{T}}}{\sigma_k^2}, \tag{10}$$

where S_b is a 23×23 matrix that sums over all the training rows in bin b. So G needs 80 Jacobians, not 274,152, and the result is identical (the code checks this against the row-by-row calculation: they agree to 10^{-15}). Likewise the prediction variance is

$$v(\boldsymbol{x}) = \boldsymbol{C}(\theta)^{\mathsf{T}} K_b\, \boldsymbol{C}(\theta), \qquad K_b = A_b H^{-1} A_b^{\mathsf{T}}, \tag{11}$$

where K_b is the 23×23 *posterior covariance of the 23 functions* a_i in bin b: a physically meaningful by-product, which gives the uncertainty of the cross section at any θ by a 23×23 quadratic form.

A consequence: at most 1,840 well-determined parameters

Since $\boldsymbol{j}$ always lies in the span of the rows of the 80 matrices A_b, G has rank at most $80 \times 23 = 1{,}840$. Whatever the amount of data, the data can determine at most 1,840 combinations of the 3,944 parameters: the 23 functions at 80 values of m_{hh}. The other 2,104 or more are fixed by the prior alone. In fact many of the 23 functions are nearly degenerate over the sampled HEFT points, and γ comes out at a few hundred (section 9). This corrects the estimate of "a few thousand" in `bnn_plan2.pdf`, which did not take the structure into account.

The coefficients C_i are obtained without touching `heftnet.py`: since f is linear in the a_i, $C_i = \partial f / \partial a_i$. The function `wilson_coefficients` gives a copy of the model a replacement `coeffs()` that returns a tensor of ones that requires a gradient, and reads C off the gradient.

6 The mixture

The ensemble gives 20 networks $\hat{\boldsymbol{\omega}}_m$, different equally good solutions. About each we build the Gaussian of eq. (5), and approximate the posterior by

$$p(\boldsymbol{\omega} \mid \mathcal{D}) \approx \sum_{m=1}^{20} \pi_m \, \mathcal{N}(\boldsymbol{\omega}; \hat{\boldsymbol{\omega}}_m, H_m^{-1}), \qquad \pi_m = \tfrac{1}{20}. \tag{12}$$

The predictive mean and variance follow from the law of total variance:

$$\bar{\mu} = \sum_m \pi_m \mu_m, \qquad s^2 = \underbrace{\sum_m \pi_m v_m}_{\text{within networks}} + \underbrace{\sum_m \pi_m (\mu_m - \bar{\mu})^2}_{\text{between networks}}. \tag{13}$$

The mean is the ensemble mean, the between term the ensemble variance; the within term is what the ensemble alone lacks: the uncertainty of each network's parameters given the data.

Why equal weights

In principle π_m is the posterior probability mass of mode m, estimated by the Laplace evidence $\ln Z_m \approx -\frac{1}{2}\chi^2_m - \frac{1}{2}\sum_i \tau_i \hat{\omega}^2_{m,i} + \frac{1}{2}\sum_g n_g \ln \tau_g - \frac{1}{2} \ln\det H_m$. In 3,944 dimensions these numbers differ by thousands between networks, and they are least reliable precisely in the directions fixed only by the prior, which dominate $\ln\det H_m$. Taken at face value they would give one network all the weight. We report them as a diagnostic and use equal weights, the assumption under which the ensemble is a fair sample of good solutions.

7 The model error

The mixture says how well networks *of this form* are determined by the data. It cannot describe the error of the form itself, the difference between the best such network and the true cross section; the χ^2 per training row of about 1.15 shows that this error exists. As in the paper's appendix, it is added to the predictive variance,

$$s^2_{\text{tot}} = s^2 + a(m_{hh})\, \sigma_k^2, \tag{14}$$

with a determined bin by bin on the 38,076 quick-test rows of the training file (never used in training) by requiring the pull $z = (t - \bar{\mu})/\sqrt{\sigma^2(1+a) + s^2}$ to have $\langle z^2 \rangle = 1$, and smoothed with $a_0 e^{-(m_{hh} - 255\,\mathrm{GeV})/\lambda} + a_1$. The 1,000 test spectra serve only to check the result.

8 Checks

1. **Exact answer** (`code/laplace_toy.py`): for a model linear in its parameters the Laplace posterior is exact. G, the prediction variances and γ agree with the textbook formulae to 10^{-16}; the covariance of 2×10^5 draws agrees with H^{-1} to the Monte Carlo precision; MacKay's iteration reaches its fixed point.
2. **Factorised versus row by row**: G and v from eqs. (10) and (11) agree with the general calculation to 10^{-15} and 10^{-13}.
3. **The linearisation**: draws from $\mathcal{N}(\hat{\omega}, H^{-1})$ evaluated with the *full* network. See section 9.
4. **The prior**: the widths with all τ_g multiplied by 0.1 and 10.

9 First results

All 20 networks of the fine-tuned bootstrap ensemble, 275,966 likelihood rows (the training rows plus the restored empty bins), about 30 seconds per network on a laptop CPU.

Each network. γ ranges from 316 to 503 (median about 410) well-determined parameters, well below the bound of 1,840 (section 5); the χ^2 per row is 1.15–1.19. The eigenvalues of H run from about 0.1 to 2×10^{12}. The Laplace evidences span 5,088 units of $\ln Z$, so evidence weights would give one network all the weight; we use equal weights, as planned.

	median width / POWHEG noise		
	all	255–555 GeV	> 555 GeV
Markov chain (`BNN_poisson_full`)	0.005	0.007	–
within networks (Laplace)	0.024	0.033	0.021
between networks (ensemble spread)	0.056	0.089	0.049
mixture, total	0.061	0.096	0.053

Table 1: The widths on the 1,000 test spectra, in units of the exact Poisson noise.

The widths (table 1, fig. 1 left). The parameter uncertainty of a single network is about 2.4% of the POWHEG noise: five times the width the Markov chain reported, which confirms that the chain explored only a small part of its own mode. It is, however, less than half the spread between networks, and adds little to it in quadrature: the mixture is 6% of the noise, the ensemble alone 5.6%.

The linearisation. Draws in all directions from $\mathcal{N}(\hat{\omega}, H^{-1})$, evaluated with the full network, scatter 5×10^4 times more than the linearisation predicts: in the directions fixed only by the prior, a parameter may move by several times its own size, far outside the region where the network is linear. In the 242 directions the data determine (eigenvalues above 100 times the largest τ_g), the full

network's spread agrees with the linearised one (ratio 1.01; 10% and 90% quantiles 0.92 and 1.13): there the approximation is accurate. These directions carry about half of v; the other half comes from weakly determined directions, where only the linearised answer is defined.

The prior. Multiplying every τ_g by 0.1 or 10 changes the median within-network width from 0.0244 to 0.0253 and 0.0233: the prior is not what sets the width.

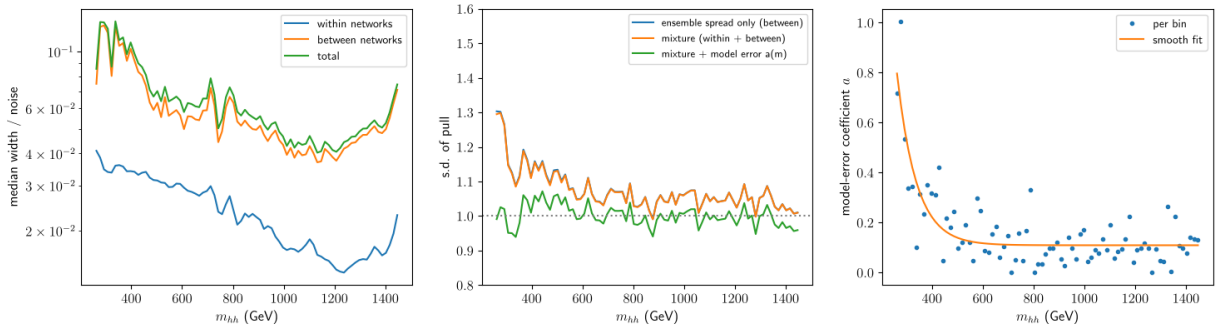


Figure 1: Left: median width of the within-network, between-network and total predictive uncertainty, in units of the noise. Middle: standard deviation of the pull on the 1,000 test spectra for the ensemble spread alone, the mixture, and the mixture with the model-error term. Right: the model-error coefficient per bin, determined on the quick-test rows, and its smooth fit.

Calibration. Without a model-error term the pulls have a standard deviation of 1.076 (1.15 near threshold), barely different from the ensemble alone (1.077): the parameter uncertainty cannot account for the misfit. With $a(m_{hh}) = 0.758\, e^{-(m_{hh}-255)/76\,\mathrm{GeV}} + 0.109$, fitted on the quick-test rows, the standard deviation is 1.003 (1.023 and 0.996 below and above 555 GeV), and 95.1% of the bins have $|z| < 2$ (95.4% for a normal distribution).

What this means for the paper

The Bayesian treatment works and tells a clear story. Given this network and these data, the parameters are well determined: about 400 effective parameters are fixed by 2.8×10^5 rows, and each network's predictions are uncertain by only 2–3% of the POWHEG noise. Different equally good networks differ by about twice that. Both are small compared with the error of the network form itself, 0.3–0.9 of the noise, which a posterior over this network's parameters cannot describe and which must enter as a model-discrepancy term. With it, the uncertainties are calibrated. The earlier Markov-chain widths are explained: the chain under-explored its mode by a factor of five in width.

10 The code

- `dihiggs/counts.py`: `recover_counts`, `recover_counts_wide`, `add_empty_bins`, `poisson_variance`.
- `dihiggs/laplace.py`: `wilson_coefficients`; the class `Laplace` with `fit_hessian` (`method='factorised'` or `'rows'`), `set_prior`, `fit_prior`, `gamma`, `predict`, `coeff_covariance`, `log_evidence`, `sample`, `evaluate`, `eigenvalues`, `state/load_state`; and `mixture`.

- `code/laplace_toy.py`: the exact-answer test.
- `code/heft_laplace.ipynb`: the whole calculation, the checks and the calibration. It runs on a laptop CPU in about 15 minutes for 20 networks and writes `runs/LAPLACE_<ensemble>/`.